

トラブルシューティング

実行時に発生する典型的なエラーと、その切り分け手順をまとめる。

エラーメッセージから原因にたどり着けないケースを中心に扱う。基本的な待機戦略の考え方は《等待策略》を、ロケータの書き方は《定位器指南》を参照。

要素が見つからない

StaleElementReferenceException

...

StaleElementReferenceException: element is not attached to the page document

...

一度取得した要素の参照が、その後の DOM 再描画によって無効になった状態。要素は画面上に「見えている」のに操作できない、という分かりにくい症状になる。

発生しやすい場面：

- 一覧の絞り込み・ソート後に、絞り込み前の行を操作しようとした
- Ajax でテーブルの中身だけ差し替わった直後
- SPA でルーティングが走り、同じ見た目の別要素になった

対処は **要素を取り直すこと**。ATP では次のステップで同じロケータを使えば自動的に再取得されるため、多くの場合は「操作の前に、その要素が属するコンテナが安定したことを確認するステップを挟む」ことで解決する。

‘SLEEP’ を入れると直ったように見えるが、再描画のタイミングは負荷によって変わるため、遅かれ早かれ再発する。規約違反でもあるので入れないこと。

ElementClickInterceptedException

...

ElementClickInterceptedException: Other element would receive the click

...

クリック対象は存在するが、**別の要素が上に重なっている**。エラーメッセージの後半に、実際にクリックを受け取ってしまう要素が出るので必ず読むこと。

よくある原因：

| 重なっているもの | 対処 |

| --- | --- |

ローディングのオーバーレイ	オーバーレイが消えるのを `WAIT\_FOR` + `ASSERT\_NOT\_EXIST` で待つ
画面下部の固定バー（同意バナー等）	先に閉じるステップを入れる。`on\_failure` は `CONTINUE`
ヘッダーの追従バー	`SCROLL\_TO` で位置を調整してからクリック

`wait\_strategy` を `CLICKABLE` にしていれば大半は自動的に待ってくれる。それでも出る場合は、重なっている要素が「消える」のではなく「ずっと居る」ケースなので、上記の対処が必要。

iframe 内の要素

要素は確かに存在するのに `ELEMENT\_NOT\_FOUND` になる場合、**iframe の中にいる**可能性を最初に疑う。

DevTools で要素を選択したとき、パンくずの先頭が `html` ではなく `#document` になっていれば iframe 内。`SWITCH\_FRAME` が必要。

メインドキュメントに戻るときは `locator\_value` に `\_\_default\_\_` を指定する。戻り忘れると、その後のステップが全部「見つからない」になる。

入力できない

日付ピッカー

`INPUT` でテキストを直接流し込んでも反映されないことがある。多くの日付ピッカーは `readonly` になっていて、キー入力を受け付けられないため。

対処は二通り：

1. **ピッカーを開いて日付セルを `CLICK` する** ——
确实だが、月をまたぐ場合はステップが増える
2. **`readonly` が付いていない実装なら `INPUT` でそのまま入る** —— 事前に DevTools で属性を確認する

書式にも注意。`YYYY/MM/DD` と `YYYY-MM-DD`

のどちらを受け付けるかは実装依存で、間違えると**エラーにならずに無視される**（値が空のまま次に進む）。入力後に `ASSERT\_TEXT` で入った値を確認するステップを入れておくと、この手の無言の失敗を防げる。

オートコンプリート付き入力欄

候補リストから選ばないと値が確定しない実装が多い。`INPUT` だけでは見た目には入力されているが、内部的には未選択のまま送信される。

`INPUT` 候補が出るのを待つ `CLICK` で候補を選択、の3ステップが必要。

全角・半角の混在

日本語環境特有の問題として、**入力値が全角に変換されてしまう**ケースがある。IMEの状態に依存する実装が原因。

数値やメールアドレスの入力欄で、`ASSERT\_TEXT`
が「見た目は同じなのに一致しない」場合はこれを疑う。ATP
の比較は前後の空白を除去するだけで、**全角・半角の正規化は行わない**。

タイムアウトする

切り分けの順序

`TIMEOUT` が出たとき、超時間を延ばす前に次を確認する：

1. **失敗時のスクリーンショットを見る** —— そもそも目的の画面に到達しているか
2. **要素は存在しているか** —— 存在するのに条件を満たさないのか、そもそも無いのか
3. **ボタンが `disabled` のままではないか** —— 前段の入力が完了していない可能性

3番が特に多い。必須項目が一つ埋まっていないだけでボタンが活性化せず、`CLICKABLE`の待機が延々と続く。この場合いくら超時間を延ばしても通らない。

待っても来ないもの

以下は待機では解決しないため、超時間の調整は無意味：

- ロケータが間違っている
- 前のステップが実は失敗している（`on\_failure: CONTINUE` で握りつぶされている）
- 非同期処理がエラーで止まっており、画面が永久に更新されない

3番目はブラウザのコンソールログで判別できる。レポートの詳細ログにコンソール出力も記録されているので確認すること。

実行結果が不安定

並行実行によるデータ競合

別々のケースが同じテストデータを操作していると、単体では通るのに一括実行では落ちる、という症状になる。

判別方法：**そのケースだけを単独実行して通るか確認する**。単独で通るなら競合の可能性が高い。

対処はテストデータを分けること。ログイン系のケースでアカウントを共有すると、片方のログアウトがもう片方のセッションを切ってしまう事故が起きやすい。

実行順による副作用

前のケースがデータを変更したまま終わっていると、次のケースが前提を満たさなくなる。

ATP はケース間の依存を持たない設計だが、**被テストシステム側の状態は共有されている**。これはプラットフォームの設計では防げないため、ケース側で「事後に元の状態へ戻す」か「毎回新しいデータを作る」必要がある。

特定の時間帯だけ落ちる

日次バッチの実行時間と重なっている可能性がある。バッチ中は一覧の表示内容が変わったり、更新がロックされたりする。

レポートの実行時刻を並べて、失敗が特定の時間帯に集中していないか確認する。集中していれば、実行タスクのスケジュールをずらすのが最も簡単な対処。

エラー分類早見表

| エラー分類 | まず疑うもの | 見るべき箇所 |

|---|---|---|

| `ELEMENT\_NOT\_FOUND` | ロケータ、iframe、ウィンドウ切替 | 失敗時スクリーンショット |

| `TIMEOUT` | ボタンの `disabled`、オーバーレイ | 直前の成功ステップの画面 |

| `ASSERT\_FAILED` | **実装側のバグの可能性が高い** | 期待値と実際値の差分 |

| `SCRIPT\_ERROR` | 実行基盤側の問題 | 再実行して再現するか |

`ASSERT\_FAILED` だけは性質が異なる。他の 3

つはケース側の不備であることが多いが、**`ASSERT\_FAILED`

は被テストシステムの不具合を捉えている可能性が高い**。安易にケースを直して緑にする前に、期待値が正しいかを必ず確認すること。